

Proyecto 3 - Modelos de grafos para la base de TWITCH (Alemania) en Pytorch Data sets usando la metodología CRISP-DM

Luis Alejandro Garzón Ramírez

Zorayda Acevedo Jiménez

Yulieth Danitza Aguillón Ortega

1. Entendimiento de la situación

Twitch es una plataforma estadounidense perteneciente a la empresa Amazon, Inc., que permite realizar transmisiones en vivo. Esta plataforma surgió en el año 2011, tiene como función principal la retransmisión de videojuegos, música y entretenimiento en general en directo, un campo en el que tiene como competidor a YouTube. De acuerdo con la página web de Twitch <https://www.twitch.tv/>, esta plataforma menciona que su misión es generar un sentido de pertenencia dándoles a los streamers las herramientas para entrelazarlos con sus seguidores. Twitch está forjando el futuro del entretenimiento interactivo en vivo, una comunidad a la vez. Actualmente, Twitch tiene más de 240 millones de usuarios activos mensuales y alrededor de 15 millones de usuarios activos diarios.

Al ser una empresa liderada por una comunidad, se comparte la imagen y visión de la plataforma abiertamente, pero hay reglas que los usuarios de plataforma deben cumplir si quieren permanecer en ella. En este contexto, uno de los grandes desafíos que enfrenta Twitch es mantener un entorno seguro y apropiado, especialmente en lo que se refiere a transmisiones de contenido para adultos o sexualmente sugerente, que va en contra de sus normas. Aunque existen mecanismos de moderación como reportes de usuarios y revisiones automáticas, muchos streamers logran evadir las reglas usando títulos engañosos, imágenes poco claras o simplemente actuando de manera ambigua. Esto complica mucho la detección a tiempo y la prevención. Adicionalmente se presenta otro reto importante ya que actualmente la plataforma tiene a los siguientes competidores en el mercado del streaming que manejan en algunos casos reglas más flexibles respecto al tema:

1. **YouTube Live (Google):** Una de las alternativas más fuertes. Muchos creadores combinan contenido grabado con transmisiones en vivo.
2. **Facebook Gaming (Meta):** Plataforma integrada en Facebook que permite a los creadores transmitir videojuegos en vivo.
3. **Kick:** Plataforma emergente creada como una alternativa “más permisiva” a Twitch, famosa por atraer a streamers con mejores ingresos.
4. **Trovo (Tencent):** Plataforma de streaming lanzada por Tencent, muy popular en algunos mercados como Asia y América Latina.
5. **DLive:** Plataforma basada en blockchain, con un sistema de recompensas descentralizado.

Todos estos competidores están intentando atraer a creadores de contenido con mejores políticas de monetización, más libertad creativa o un mejor soporte técnico. La guerra por los streamers es evidente, y Twitch, aunque sigue siendo el líder, ha perdido exclusividad con algunos creadores importantes en los últimos años.

Con esos retos se busca a través de los datos publicados en Pytorch Datasets construir un modelo de grafos que prediga si un usuario en este caso que pertenezca al país de Alemania tiene altas probabilidades de transmitir contenido para adultos o no, incluso antes de que lo haga de forma abierta. Para lograr esto, usaremos modelos llamados redes neuronales de grafos (Graph Neural Networks o GNNs), entrenadas con herramientas de PyTorch, una de las bibliotecas más robustas en el campo. Los grafos en este tipo de problemas son útiles ya que aprenden de los datos estructurados como redes, en este caso esta plataforma de streaming actúa como una, ya que tenemos usuarios y seguidores relacionados entre sí. En lugar de analizar solo las características de un objeto, se tienen en cuenta también las relaciones entre los objetos.

De acuerdo con la página de donde se extraen los datos para el análisis realizado: <https://pytorch-geometric.readthedocs.io/> esta fuente contiene las redes de jugadores de Twitch se presentaron en el artículo "Integración de nodos atribuidos a múltiples escalas". Los nodos representan a los jugadores en Twitch y los bordes representan los seguidores entre ellos. Las características de los nodos representan las incrustaciones de los juegos jugados por los usuarios de Twitch. Este proyecto combina el análisis de redes sociales con herramientas avanzadas de analítica de datos para resolver un problema real y sensible, ayudando a que plataformas como Twitch a que sean más seguras para todos.

2. Objetivo de negocio

Identificar de forma anticipada a los usuarios de Twitch en Alemania que probablemente transmitan contenido para adultos, con el fin de fortalecer los mecanismos de reporte temprano, para proteger la comunidad y asegurar el cumplimiento de las políticas de la plataforma.

$$KPI = \frac{\text{Total de usuarios predichos como transmisores de contenido para adultos}}{\text{Usuarios correctamente identificados como transmisores de contenido para adultos}} \times 100$$

3. Objetivo de minería de datos

El principal objetivo para la plataforma es asegurarse de que el contenido presentado sea apropiado, pero a su vez es importante no generar alarma sobre contenido que no infringe las normas, ya que esto podría generar bloqueos o acciones sobre usuarios sin este contenido, los cuales terminen considerando otras plataformas y así perder mercado.

En ese orden de ideas tenemos como objetivo generar un modelo de clasificación que permita obtener los siguientes valores de métricas en el conjunto de datos de test:

- Precisión clase objetivo > 0,8
- Recall clase objetivo > 0,8

4. Entendimiento de los datos

El conjunto de datos contiene un solo grafo, compuesto por: Número de nodos: 9,498, Número de características por nodo: 128 y Número de clases: 2 (es decir, existen dos posibles etiquetas para clasificar los nodos, en este caso publica o no contenido para adultos)

En cuanto a la estructura del grafo, se observa lo siguiente: Las aristas no son dirigidas, lo que indica que las conexiones entre nodos son bidireccionales, el grafo no contiene nodos aislados, es decir, todos los nodos están conectados al menos a otro nodo y el grafo sí contiene bucles, lo que significa que algunos nodos están conectados consigo mismos, en este caso de negocio puede el mismo usuario seguirse a sí mismo.

Hay dos tipos de nodos en el grafo. La clase 0 tiene 3756 nodos y la clase 1 tiene 5742 nodos. Esto significa que hay más nodos de la clase 1 que de la clase 0.

Estadísticas del grado (número de conexiones de cada nodo): El promedio de conexiones por nodo es 34.25, es decir, en promedio, cada nodo está conectado a unas 34 personas, el nodo con más conexiones tiene 4261 conexiones (muy conectado), el nodo con menos conexiones tiene 3 conexiones, la densidad del grafo es 0.003606, lo que indica que hay pocas conexiones en relación con todas las posibles (el grafo es bastante disperso). Hay 603,088 triángulos, que son grupos de tres nodos donde cada uno está conectado con los otros dos, mostrando estructuras cerradas o grupos en el grafo.

El coeficiente de clustering promedio es 0.2009, que mide que tan conectados están los vecinos de un nodo entre sí. Un valor moderado, indicando que hay cierta formación de grupos. La transitividad global es 0.0465, que es otra medida relacionada con la cantidad de triángulos en el grafo, indicando que las conexiones en triángulos no son muy frecuentes a nivel global. Hay 1 componente conectado, lo que significa que todos los nodos están conectados directa o indirectamente en un solo grupo grande, y por último, el componente más grande tiene 9498 nodos, prácticamente todo el grafo.

Centralidad por grado: mide qué tan conectado está un nodo: El nodo 7787 es el más conectado (con centralidad 0.4487). Los siguientes nodos más conectados tienen valores menores (0.1924, 0.1809, etc.).

Centralidad por intermediación (betweenness): mide cuántas veces un nodo actúa como puente o paso en los caminos más cortos entre otros nodos.

- El nodo 7787 también tiene el valor más alto (0.2912), siendo clave para la comunicación en la red.
- Los demás nodos tienen valores más bajos, pero siguen siendo importantes.

Revisamos la eficiencia global de los datos, la cual es de 0,3891 el valor está en un rango de 0 a 1. Un valor cercano a 0 indica una red fragmentada o mal conectada: muchos nodos están lejanos o incluso desconectados. En un grafo de red social como es el caso de Twitch, vemos que las personas están conectadas, pero no de forma óptima, es decir, la información no se propaga tan fácilmente como en una red completamente interconectada.

Por último, observamos que la asorsatividad de los datos mide cómo se conectan los nodos en una red según alguna propiedad. Un resultado de -0.1095 como fue el resultado de los datos de Twitch este valor mide si los nodos con un alto grado (muchas conexiones) tienden a conectarse con otros también de alto grado (positivo), o si más bien se conectan con nodos de bajo grado (negativo). Como es un valor ligeramente negativo, lo que sugiere que los

nodos con muchas conexiones tienden a conectarse con nodos con pocas conexiones, esto suceden principalmente en redes sociales donde la persona que sigue a los que publican contenido tienen menos seguidores comparado con los que publican el contenido.

5. Preparación de los datos

Normalización de los datos: Se realiza la normalización de `data.x` teniendo en cuenta que sus vectores tienden a tener valores muy cercanos a cero, estos valores pueden ocasionar que predominen los nodos más grandes haciendo los modelos inestables, antes de normalizar los datos se encontraban en esta escala:

```
Min value: 0.0000
Max value: 0.1006
Mean: 0.0078
Norma promedio: 0.1265
```

De acuerdo a la documentación de la librería de Pytorch Geometrics trae la función `NormalizeFeatures` la cual realiza esta tarea, mas sin embargo, no se logró obtener los resultados esperados, por lo que se procedió a realizar la normalización de manera manual, `data.x = F.normalize(data.x, p=2, dim=1)`, obteniendo resultados satisfactorios y filas aproximadas a valor de 1.

```
Min value: 0.0000
Max value: 0.6859
Mean: 0.0620
Norma promedio: 1.0000
```

Creación subconjuntos de train, val y test: Las divisiones para realizar el entrenamiento, validación y prueba se realizaron con porcentajes del dataset de 70% para entrenamiento, 10% validación y 20% para test estableciéndose las siguientes cantidades de nodos por división:

```
Nodos de entrenamiento: 6648
Nodos de validación: 950
Nodos de prueba: 1900
```

6. Modelos

Modelo 0

Se utiliza como modelo base Graph Neural Network Multilayer Perceptron, el cual utiliza como entrada los nodos del grafo y adicional a ello las variables `x` con la modificación realizada en la preparación de los datos, se usa como función de perdida `BCEWithLogitsLoss` y función sigmoide en las predicciones al ser un problema binario, el modelo tiene una primera capa lineal, una capa de dropout para reducir el sobreajuste y una capa lineal con dimensión de salida de uno, la estructura es la siguiente:

```
MLP(
  (linear1): Linear(in_features=128, out_features=16, bias=True)
  (bn1): BatchNorm1d(16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (dropout): Dropout(p=1, inplace=False)
  (linear2): Linear(in_features=16, out_features=1, bias=True)
```

El modelo se entrena en 100 épocas con un `with_decay` de $1e-3$ para evitar sobre ajuste, el resultado en el dataset de test es un recall de 1 con precisión de 0,61 y AUC de 0,42, esto indica que el modelo es peor que el azar en la predicción y sucede ya que al hacer la predicción recae sobre la clase mayoritaria, es decir el valor de 1.

	Precision	Recall	F1-score	Support
0.0	0.00	0.00	0.00	740
1.0	0.61	1	0.76	1160
Accuracy			0.61	1900
Macro avg	0.31	0.50	0.38	1900
Weighted avg	0.37	0.61	0.46	1900

6.1 Modelo 1 GNN matriz de adyacencia

El modelo 1 (GNN) utiliza una matriz de adyacencia densa y normalizada, lo cual permite controlar la influencia de los nodos con alto grado (es decir, con mayor número de conexiones), evitando que dominen la propagación de información. Además, se incorporan self-loops (conexiones de los nodos consigo mismos) para asegurar que, durante la agregación de información desde los vecinos, cada nodo también conserve parte de su propia información. Esta estrategia mejora la estabilidad del modelo y permite un aprendizaje más equilibrado entre las representaciones locales y estructurales del grafo.

Hiperparametros:

- Número de unidades ocultas: 32
- Tasa de aprendizaje: 0.01
- Weight decay: $5e-4$
- Función de perdida: CrossEntropyLoss
- Número de épocas: 150

Se implementaron 2 capas lineales aplicadas a los vectores de características de los nodos, entre ambas capas se utilizó la función de activación Relu para que el modelo aprenda relaciones complejas en los datos y se aplicó dropout de 0.5 ayudando a reducir el overfitting. En la etapa de entrenamiento el modelo realiza en cada paso de propagación una multiplicación entre la matriz de adyacencia y el `data.x` (características de los nodos) integrando así información del vecindario robusteciendo el aprendizaje del modelo.

Resultados en entrenamiento, validación y test:

Train

```
Epoch 0 | Train Loss: 0.676 | Train Acc: 60.36%
Epoch 20 | Train Loss: 0.656 | Train Acc: 60.36%
Epoch 40 | Train Loss: 0.632 | Train Acc: 65.00%
Epoch 60 | Train Loss: 0.608 | Train Acc: 66.86%
Epoch 80 | Train Loss: 0.599 | Train Acc: 68.19%
Epoch 100 | Train Loss: 0.591 | Train Acc: 68.92%
Epoch 120 | Train Loss: 0.590 | Train Acc: 68.80%
Epoch 140 | Train Loss: 0.589 | Train Acc: 67.72%
```

Validacion

```
Resultados del modelo GNN:
Accuracy : 65.68%
AUC      : 0.717
Matriz de Confusión:
[[138 243]
 [ 83 486]]

Reporte de Clasificación:
      precision    recall  f1-score   support

0         0.624      0.362      0.458       381
1         0.667      0.854      0.749       569

 accuracy          0.657       950
 macro avg         0.646      0.608      0.604       950
 weighted avg      0.650      0.657      0.632       950

GNN validation accuracy: 65.68%
GNN validation AUC: 0.717
```

Test

```

Resultados del modelo GNN:
Accuracy : 67.37%
AUC      : 0.726
Matriz de Confusión:
[[293 447]
 [173 987]]

Reporte de Clasificación:
      precision    recall  f1-score   support

     0       0.629       0.396       0.486        740
     1       0.688       0.851       0.761       1160

 accuracy          0.659          0.623          0.623        1900
 macro avg          0.659          0.623          0.623        1900
 weighted avg          0.665          0.674          0.654        1900

GNN test accuracy: 67.37%
GNN test AUC: 0.726

```

Si bien el modelo logró clasificar correctamente a la mayoría de los usuarios que transmiten contenido para adultos, presentó dificultades para identificar la clase minoritaria (usuarios que no transmiten contenido para adultos), evidenciado en un bajo valor de recall para esta clase en los tres conjuntos de datos evaluados. Esto indica una alta inclinación del modelo hacia la clase mayoritaria (clase 1). A pesar de ello, el modelo demuestra una capacidad efectiva para generalizar, aprovechando tanto la topología del grafo como las características de los nodos. Sin embargo, este comportamiento sesgado refleja el impacto del desbalance de clases presente en el dataset.

6.2 Modelo 2 GCN convolucional

Se implementa y entrena un modelo GCN (Graph Convolutional Network) para una tarea de clasificación binaria de nodos (publica o no contenido para adultos en Twitch Alemania) usando PyTorch Geometric. Algunos Hiperparámetros importantes son:

Nombre Hiperparámetro	Comentario
hidden_channels=16	Número de neuronas en la capa oculta
lr=0.01	Tasa de aprendizaje del optimizador Adam
weight_decay=5e-4	Regularización L2 (evita sobreajuste)
dropout p=0.5	Probabilidad de "apagar" neuronas (solo en entrenamiento)
BCEWithLogitsLoss()	Función de pérdida usada
epochs=200	Número de épocas de entrenamiento
optimizer = Adam	Optimizador usado
threshold = 0.5 implícito en probs >= 0.5	Umbral de decisión para clasificación

Elementos relevantes del modelo GCN:

- Dos capas GCN (GCNConv)
 - Primera capa: proyecta características a hidden_channels
 - Segunda capa: salida de 1 neurona → predicción binaria
- Función de activación: ReLU después de la primera capa
- Regularización: Dropout aplicado entre capas
- Salida: view(-1) convierte la salida a un vector 1D

Resultados en el conjunto de prueba:

- **Test Accuracy:** 0.66. Significa que el modelo clasificó correctamente el 66% de los nodos (o ejemplos) del conjunto de prueba. Es decir, de cada 100 predicciones que hizo el modelo, 66 fueron correctas (ya sea verdaderos positivos o verdaderos negativos).
- **Test AUC:** 0.72. indica qué tan bien el modelo distingue entre clases (positiva y negativa). AUC mide el área bajo la curva ROC, que compara la tasa de verdaderos positivos (TPR) frente a la tasa de falsos positivos (FPR) a diferentes umbrales de decisión. En este caso, 0.72 significa que hay un 72% de probabilidad de que el modelo le dé una puntuación más alta a un nodo positivo que a uno negativo.

Matriz de Confusión:

- Verdaderos negativos, 244, El modelo dijo 0 y era 0 (acertó con negativos)
- Falsos positivos, 496, El modelo dijo 1 pero era 0 (se equivocó, falso 1)
- Falsos negativos, 148, El modelo dijo 0 pero era 1 (se equivocó, falso 0)
- Verdaderos positivos (TP), 1012, El modelo dijo 1 y era 1 (acertó con positivos)

	Predicho 0	Predicho 1
Verdadero 0	244	496
Verdadero 1	148	1012

Reporte de Clasificación: se puede observar en los datos de Clase 0 (negativos) lo siguiente:

- Precision, 0.62, De los que el modelo dijo que eran 0, el 62% eran correctos
- Recall, 0.33, De todos los reales 0, solo el 33% fueron detectados
- F1-score, 0.43, Promedio entre precisión y recall (bajo)
- Support, 740, Total de ejemplos reales de clase 0

Se concluye con los datos anteriores, que el modelo tiene dificultad para detectar correctamente la clase 0. En las métricas para la Clase 1 (positivos) se observa que:

- Precision, 0.67, De los que el modelo dijo que eran 1, el 67% eran correctos
- Recall, 0.87, Detecta correctamente el 87% de los positivos
- F1-score, 0.76, Buen equilibrio entre precisión y recall
- Support, 1160, Total de ejemplos reales de clase 1

	Precision	Recall	F1-score	Support
0.0	0.622449	0.329730	0.431095	740.000000
1.0	0.671088	0.872414	0.758621	1160.000000
Accuracy	0.661053	0.661053	0.661053	0.661053
Macro avg	0.646768	0.601072	0.594858	1900.000000
Weighted avg	0.652144	0.661053	0.631058	1900.000000

En general el modelo detecta bien la clase 1, pero a costa de fallar más en la clase 0.

6.3 Modelo 3 GAT atención

Dado el descubrimiento del modelo base donde se ve una tendencia del modelo a clasificar los nodos según la clase mayoritaria se inicia cambiando un poco la función de perdida al usar BCEWithLogitsLoss con ponderación positiva según pesos previamente definidos, estos pesos se calculan dada la relación entre clases del dataset de train, la idea del GAT es que pueda ponderar los pesos de los nodos según su relevancia lo cual se hace especialmente importante recordando que en la descripción de los datos la mayoría de nodos tienen un grado bajo y existen unos muy atípicos (con 4.261 conexiones por ejemplo).

Durante la ejecución del modelo, denotamos que los resultados estaban siendo bajos aun haciendo iteraciones con varios ajustes, por lo que adicional a los nodos y sus características normalizadas, se incluyó un embedding de los nodos, este fue realizado con la librería Node2Vec y sus hiperparametros fueron ajustados con Optuna para que el embedding tuviese una división apropiada.

Finalmente, la red se compone de tres bloques principales de atención utiliza un dropout de 0,45 para evitar overffiting, se usa un treshold de 0,45 el cual es relativamente bajo dado el desbalance de clases y la importancia del recall como métrica objetivo, se utiliza un learning rate de 0.0001361 y weight_decay=0.00001, el modelo se entrena durante 500 epocas, con 6 cabezas de atención.

	Precision	Recall	F1-score	Support
0.0	0.62	0.45	0.52	740
1.0	0.70	0.83	0.76	1160
Accuracy			0.68	1900
Macro avg	0.66	0.63	0.64	1900
Weighted avg	0.67	0.68	0.66	1900

El resultado final es un AUC de 0,72 lo que representa una alta mejoría respecto al modelo base, accuracy de 0,68 el cual presenta una leve mejora siendo que el modelo aún tiene baja efectividad en la clase minoritaria, por último, precision de 0,70 y recall de 0,83 en la clase objetivo los cuales son valores muy buenos y cercanos al objetivo siendo que el recall se cumple, pero la precision aun representa una oportunidad de mejora (aunque dicho sea mucho menor que la del modelo base).

7. Evaluación

A continuación, se presenta de forma consolidado los resultados de los modelos expuestos anteriormente:

Modelo GNN	AUC	Accuracy	Precision		Recall	
			0	1	0	1
Modelo 0 - Nodes with features	0,49	0,61	0	0,61	0	1
Modelo 1 - Matriz de adyacencia	0,72	0,67	0,62	0,68	0,39	0,85
Modelo 2 - Convolutacional	0,72	0,66	0,62	0,67	0,39	0,87
Modelo 3 - Attention	0,72	0,68	0,62	0,70	0,45	0,83

El mejor modelo para cumplir con el objetivo de negocio y minería de datos fue el GAT con node2vec y balanceo, aunque su ventaja recae probablemente en la elección del threshold o punto de corte ya que para la GCN y GAT se cumple un mismo nivel de AUC que es la capacidad para identificar las clases positiva y negativa, bajo este contexto se puede satisfacer el objetivo de identificar una proporción considerablemente alta de los usuarios que transmiten contenido para adultos.

Sin embargo, existe la necesidad de mejorar el grafo obtenido ya sea añadiendo más características de los usuarios o incorporando mejores conexiones (mejorar eficiencia global) entre los nodos ya que si bien se mejora considerablemente la precisión del modelo respecto al modelo base, esta medida no logra satisfacer la necesidad implícita del negocio, es decir aún hay un porcentaje considerable de los que clasifica como usuarios con contenido inapropiado que realmente no transmiten dicho contenido, se harán recomendaciones al respecto en la siguiente sección.

8. Recomendaciones

De acuerdo a los resultados de cada modelo, se recomienda implementar el **modelo 03 – GAT**, teniendo en cuenta que si bien las métricas son similares en los modelos 1, 2 y 3 el modelo 3 se distingue por tener la mayor precisión (0.71) y el mayor recall (0.83), con base en este modelo, la plataforma Twitch podría generar un sistema de alertas tempranas considerando que se podrían identificar patrones comunes de jugadores que emitan contenido para adultos de manera anticipada y centrando la atención y protocolos de seguimiento a estas cuentas para identificar si están incumpliendo las normas de emisión de contenido en la plataforma.

Así mismo detectando de manera temprana usuarios sospechosos se pueden tomar acciones con estas cuentas para no exponer a los usuarios legítimos de la plataforma a visualización de contenido para adulto creando estrategias de penalización y/o sanciones o bloqueos parciales a cuentas que se detecten infractoras y premiando con bonos o acceso a privilegios a usuarios que utilizan la plataforma de manera correcta.

Se recomienda a Twitch enriquecer los datos con características como frecuencias de transmisión, tipología de audiencia, descripciones con el fin de poder mejorar la precisión del modelo y reducir la tasa de falsos positivos.

Referencias

PyTorch Geometric. (s.f.). *torch_geometric.datasets.Twitch*. https://pytorch-geometric.readthedocs.io/en/2.5.3/generated/torch_geometric.datasets.Twitch.html

Twitch. (s.f.). *Acerca de*. <https://www.twitch.tv/p/es-mx/about/>

Anexo.

A. Reporte de trabajo en equipo

Nº	Actividad asignada	Responsable*	Fecha inicio	Fecha Fin
0	Selección de los datos: Bases de Pytorch data sets	Zorayda Acevedo Yulieth Aguillón Alejandro Garzón	06/05/2025	06/05/2025
1	Entendimiento de la situación	Yulieth Aguillón	11/05/2025	11/05/2025
2	Objetivo de negocio	Yulieth Aguillón	11/05/2025	11/05/2025
3	Objetivo de minería de datos	Alejandro Garzón	18/05/2025	18/05/2025
4	Entendimiento de los datos	Yulieth Aguillón	17/05/2025	17/05/2025
5	Preparación de los datos	Zorayda Acevedo	17/05/2025	17/05/2025
6	Modelo 0 – Línea base	Alejandro Garzón	18/05/2025	18/05/2025
6.1	Modelo 1 – Adjacency Matrix	Zorayda Acevedo	17/05/2025	17/05/2025
6.2	Modelo 2 – Graph Convolutional Networks (GCN)	Yulieth Aguillón	18/05/2025	18/05/2025
6.3	Modelo 3 - GAT	Alejandro Garzón	18/05/2025	18/05/2025
7	Evaluación	Alejandro Garzón	19/05/2025	19/05/2025
8	Recomendaciones	Zorayda Acevedo	19/05/2025	19/05/2025
9	Código en Colab	Zorayda Acevedo Yulieth Aguillón Alejandro Garzón	20/05/2025	20/05/2025
10	Presentación proyecto 3 (ppt)	Zorayda Acevedo Yulieth Aguillón Alejandro Garzón	20/05/2025	20/05/2025
11	Cargue de los soportes en GitHub	Zorayda Acevedo	23/05/2025	23/05/2025
12	Sustentación proyecto	Zorayda Acevedo Yulieth Aguillón Alejandro Garzón	24/05/2025	24/05/2025

*Persona quien realiza la actividad mencionada